VOLUME 02 OF 10

Mathematics for Games and Computing

Undergraduate mathematical core

Build the mathematical language for algorithms, graphics, physics, signals and experiments.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 06 Discrete mathematical reasoning

06.1 Algebra, functions, logarithms and trigonometry • 06.2 Logic, sets and quantified claims • 06.3 Proofs, induction and invariants • 06.4 Counting, graphs and recurrences

Module 07 Linear algebra and game geometry

07.1 Vectors, dot products and projections • 07.2 Matrices, bases and linear systems • 07.3 Homogeneous transforms, cameras and quaternions • 07.4 Eigenvalues, least squares and singular values

Module 08 Calculus, mechanics and numerical simulation

08.1 Limits, derivatives and rates • 08.2 Integrals, accumulation and continuous probability • 08.3 Newtonian mechanics and differential equations • 08.4 Time integration, stability and error

Module 09 Probability, statistics and evidence

09.1 Probability and conditional reasoning • 09.2 Random variables, distributions and sampling • 09.3 Inference, uncertainty and benchmark design • 09.4 Regression, correlation and ethical telemetry

Module 10 Signals, optimization and numerical foundations

10.1 Multivariable calculus and gradients • 10.2 Sampling, aliasing and Fourier ideas • 10.3 Convolution, filters and systems • 10.4 Optimization, conditioning and approximation budgets

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 06

Discrete mathematical reasoning

Prerequisites Volume 01; complete the algebra diagnostic in START_HERE before proceeding.

Four lessons

06.1 Algebra, functions, logarithms and trigonometry

Translate a game relationship into an equation with a defined domain.

06.2 Logic, sets and quantified claims

Express requirements precisely enough to test or prove.

06.3 Proofs, induction and invariants

Justify a program property instead of testing only examples.

06.4 Counting, graphs and recurrences

Model combinations and recursive work without brute-force guessing.

Module project

Write a mathematical notebook containing a truth-table specification, an induction proof, a graph model and a justified operation-count estimate. Include counterexamples to three false claims.

Core references

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

R08 MIT OpenCourseWare — 18.01SC Single Variable Calculus

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 06.1 • MODULE 06

Algebra, functions, logarithms and trigonometry

Learning goal Translate a game relationship into an equation with a defined domain.

A function maps each allowed input to one output. Its domain matters: dividing by zero or taking a real square root of a negative number is not repaired by notation. Rearranging an equation must preserve its conditions. Exponents model repeated multiplication; logarithms invert exponentiation. log2(n) often counts how many times a size can be halved.

Angles connect geometry to periodic motion. In radians, a full revolution is 2*pi. For a unit circle, a direction at angle t has components (cos(t), sin(t)). Check units before calling a trigonometric function. Degrees and radians are different representations of the same angle, not interchangeable numeric inputs.

WORKED EXAMPLE

At 90 degrees, or pi/2 radians, the unit-circle direction is (0, 1). A list of 1024 items can be halved ten times to reach one because 2^10 = 1024.

PRACTICE + PASS CRITERION

Solve distance = speed*time for each variable and state forbidden inputs. Convert 0, 90, 180 and 360 degrees to radians. Pass: solve ten algebra problems and explain logarithms using repeated halving before advancing.

ASSIGNED STUDY

R08: prerequisite algebra review and trigonometric functions; R06: sums and asymptotics.

CHECK YOUR UNDERSTANDING

1. What is log2(64)?

2. Convert 180 degrees to radians.

3. What restriction applies when computing speed = distance/time?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 06.2 • MODULE 06

Logic, sets and quantified claims

Learning goal Express requirements precisely enough to test or prove.

Propositional logic combines true/false statements. Implication P -> Q fails only when P is true and Q is false. It does not claim that P actually happens. A universal claim says a property holds for every allowed input; an existential claim says at least one input has it. Negating a universal claim requires one counterexample.

Sets group distinct elements; relations describe connections; functions are relations with exactly one output for each domain element. These ideas support requirements such as every live entity has one valid transform. Ambiguous language like "usually valid" cannot serve as an invariant until its exceptions are specified.

WORKED EXAMPLE

"Every saved entity has an ID" is disproved by one saved entity without an ID. "Some entity is visible" requires only one witness and does not imply all entities are visible.

PRACTICE + PASS CRITERION

Formalize five requirements for the grid game using sets and quantifiers. Construct truth tables for a door-access rule. Pass: provide a counterexample or witness for each negated requirement.

ASSIGNED STUDY

R06: propositions, predicates, sets and relations.

CHECK YOUR UNDERSTANDING

1. When is P -> Q false?

2. Negate "every unit has positive health."

3. Why must the domain of a quantified statement be explicit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 06.3 • MODULE 06

Proofs, induction and invariants

Learning goal Justify a program property instead of testing only examples.

A proof is a chain of justified steps from assumptions to a conclusion. Direct proof, contradiction and induction solve different kinds of questions. Induction establishes a base case and shows that validity at one step implies validity at the next. Both parts are necessary.

A loop invariant connects mathematical reasoning to implementation: establish it before the loop, preserve it in each iteration, and combine it with the exit condition to prove the result. Testing complements proofs by checking whether code matches the modeled assumptions. A proof of an ideal algorithm does not automatically establish the correctness of its floating-point implementation.

WORKED EXAMPLE

To prove the running sum after k iterations equals 1+...+k, start at k=0 with sum=0. If the claim holds at k, adding k+1 produces the required sum for k+1.

PRACTICE + PASS CRITERION

Prove the formula 1+...+n = n(n+1)/2 by induction. Then write an invariant for counting living units in a prefix of a list. Pass: show initialization, preservation and termination explicitly.

ASSIGNED STUDY

R06: induction, invariants and proof methods.

CHECK YOUR UNDERSTANDING

1. What are the two essential parts of induction?

2. What is 1+...+100?

3. Why does a mathematical proof not eliminate implementation tests?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 06.4 • MODULE 06

Counting, graphs and recurrences

Learning goal Model combinations and recursive work without brute-force guessing.

The product rule counts independent sequential choices; the sum rule counts disjoint alternatives. A graph represents vertices and edges, which may be directed or weighted. A level map, dependency graph and network topology are different applications of the same abstraction. Decide what an edge means before choosing an algorithm.

A recurrence describes a quantity in terms of smaller instances. Repeatedly halving a search space gives logarithmic depth, but doing linear work at every level changes total cost. Do not confuse possible states with states actually explored by a particular algorithm. Counting provides upper bounds and exposes when exhaustive search becomes unrealistic.

WORKED EXAMPLE

Four independent equipment slots with three choices each give 3^4 = 81 combinations. An undirected simple graph with n vertices has at most n(n-1)/2 edges because each unordered pair appears once.

PRACTICE + PASS CRITERION

Build a dependency graph for game startup and identify an invalid cycle. Count loadout combinations with and without a restriction. Pass: explain the recurrence for halving a search range and distinguish depth from total work.

ASSIGNED STUDY

R06: counting, graphs, recurrences and asymptotic notation.

CHECK YOUR UNDERSTANDING

1. How many combinations arise from five independent binary choices?

2. How many edges can a simple undirected graph with five vertices have?

3. Why is a dependency cycle a problem for a simple startup order?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 07

Linear algebra and game geometry

Prerequisites Module 06.1; basic Python functions.

Four lessons

07.1 Vectors, dot products and projections

Use vectors for direction, distance and component measurements.

07.2 Matrices, bases and linear systems

Understand a matrix as a transformation rather than a table to memorize.

07.3 Homogeneous transforms, cameras and quaternions

Compose a scene using explicit coordinate conventions.

07.4 Eigenvalues, least squares and singular values

Recognize stability and approximation problems in game data.

Module project

Build a tested vector/matrix library for a two-dimensional scene, then demonstrate a three-dimensional camera transformation with written coordinate conventions.

Core references

R07 MIT OpenCourseWare — 18.06 Linear Algebra

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 07.1 • MODULE 07

Vectors, dot products and projections

Learning goal Use vectors for direction, distance and component measurements.

A vector represents a magnitude and direction in a chosen coordinate system. A point represents a location; subtracting points yields a displacement vector. The Euclidean length of (x,y,z) is sqrt(x*x+y*y+z*z). Normalization divides by length, so the zero vector needs a deliberate policy.

The dot product measures alignment and produces a scalar. For unit vectors it equals the cosine of the included angle. Projection onto a unit direction u is dot(v,u)*u. In three dimensions the cross product produces a perpendicular vector with orientation determined by the coordinate convention. Do not use a cross-product sign without declaring handedness.

WORKED EXAMPLE

For v=(3,4), length is 5 and its unit direction is (0.6,0.8). Dotting v with u=(1,0) gives 3, the component along the x axis.

PRACTICE + PASS CRITERION

Implement length, dot and safe normalization. Use dot products to determine whether a target is in front of a character. Pass: test zero vectors, perpendicular vectors and opposite directions.

ASSIGNED STUDY

R07: vectors and projections; R23: Geometry and Transformations / Vectors.

CHECK YOUR UNDERSTANDING

1. What is the length of (6,8)?

2. What is the dot product of (1,0) and (0,1)?

3. Why cannot the zero vector be normalized by the ordinary formula?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 07.2 • MODULE 07

Matrices, bases and linear systems

Learning goal Understand a matrix as a transformation rather than a table to memorize.

A matrix describes how basis vectors are mapped. Matrix-vector multiplication combines those images to transform a vector. A basis is a set of independent directions spanning the space. Changing basis changes coordinates without necessarily changing the underlying geometric object.

Matrix multiplication composes transformations, and order generally matters. Solving Ax=b is not the same task as explicitly computing A inverse; elimination or factorization is often preferable. A singular matrix loses information and cannot be uniquely inverted. In graphics, a zero scale along an axis is a concrete example of collapsing multiple points to the same location.

WORKED EXAMPLE

In two dimensions, scaling x by 2 maps (1,1) to (2,1). Rotating and then applying a nonuniform scale generally differs from applying the scale and then rotating.

PRACTICE + PASS CRITERION

Implement 2-by-2 multiplication and solve two small linear systems by elimination on paper. Pass: show a pair of transforms that do not commute and explain a singular example geometrically.

ASSIGNED STUDY

R07: matrix multiplication, elimination, inverses and vector spaces.

CHECK YOUR UNDERSTANDING

1. Do matrices generally satisfy AB = BA?

2. What happens to invertibility when an axis is scaled to zero?

3. Why solve Ax=b without explicitly forming an inverse?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 07.3 • MODULE 07

Homogeneous transforms, cameras and quaternions

Learning goal Compose a scene using explicit coordinate conventions.

Homogeneous coordinates allow translation to be represented with matrix multiplication. In a common three-dimensional convention, points use w=1 and directions use w=0. A model transform maps local coordinates into world space; a view transform maps world coordinates into camera space. Perspective projection introduces a division by w and changes how depth is represented.

Choose row or column vectors, multiplication order, handedness and depth range, then document them. A unit quaternion represents orientation compactly and supports interpolation, but multiplication order still matters and accumulated error may require normalization. Euler angles remain useful for interfaces; their limitations do not make them meaningless.

WORKED EXAMPLE

With column vectors and a local-to-world matrix M, a point is p_world=M*p_local. A parent-child chain is M_parent*M_child*p_local. Reversing it changes the coordinate interpretation.

PRACTICE + PASS CRITERION

Create a parented object and camera using documented conventions. Pass: test identity, inverse round trips within tolerance, a translated direction that remains unchanged, and a parent rotation that moves its child correctly.

ASSIGNED STUDY

R23: Coordinate Systems and Transformations; R19: geometric transformations.

CHECK YOUR UNDERSTANDING

1. Why commonly use w=0 for directions?

2. What does the view transform do?

3. Why document multiplication order even when using a library?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 07.4 • MODULE 07

Eigenvalues, least squares and singular values

Learning goal Recognize stability and approximation problems in game data.

An eigenvector keeps its direction under a linear transformation; its eigenvalue is the scale applied along that direction. These concepts help analyze repeated updates and dynamical systems. Least squares fits an approximate solution when equations are inconsistent or measurements are noisy. It minimizes a chosen squared residual, not necessarily every meaningful error.

Singular values describe how strongly a matrix stretches different directions. Very small singular values can make inversion sensitive to input errors. Condition number measures sensitivity of the problem; algorithmic stability concerns how a method handles arithmetic error. A well-written algorithm cannot remove information absent from a badly conditioned problem.

WORKED EXAMPLE

The diagonal matrix diag(2,0.5) stretches the x direction by 2 and y by 0.5. Repeated application expands one component while shrinking the other.

PRACTICE + PASS CRITERION

Fit a line to five noisy position/time observations and report residuals. Analyze repeated multiplication by a diagonal matrix. Pass: distinguish data noise, poor conditioning and a programming error in your report.

ASSIGNED STUDY

R07: eigenvalues, least squares and singular value decomposition.

CHECK YOUR UNDERSTANDING

1. What are the eigenvalues of diag(3,2)?

2. What objective does ordinary least squares minimize?

3. Can a stable algorithm make an ill-conditioned problem insensitive?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 08

Calculus, mechanics and numerical simulation

Prerequisites Modules 06.1 and 07.1; work through the linked calculus problem sets.

Four lessons

08.1 Limits, derivatives and rates

Connect derivatives to velocity, acceleration and local approximation.

08.2 Integrals, accumulation and continuous probability

Interpret integration as accumulated contribution.

08.3 Newtonian mechanics and differential equations

Model motion from forces with clear simplifying assumptions.

08.4 Time integration, stability and error

Choose and test a numerical update rather than assuming smaller code is correct.

Module project

Build a projectile and spring simulator with analytic comparison cases, timestep refinement, unit checks and a report separating model error from numerical error.

Core references

R08 MIT OpenCourseWare — 18.01SC Single Variable Calculus

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

R28 Box2D project — Box2D documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 08.1 • MODULE 08

Limits, derivatives and rates

Learning goal Connect derivatives to velocity, acceleration and local approximation.

A derivative measures the local rate of change of a function. Velocity is the derivative of position with respect to time; acceleration is the derivative of velocity. The derivative carries units: meters differentiated by seconds gives meters per second. Product and chain rules explain how composed calculations change.

A finite difference approximates a derivative from samples. Making the step smaller reduces some approximation errors but can worsen floating-point cancellation. The derivative of an ideal formula and the difference between two measured samples answer related but not identical questions. Always distinguish the mathematical function from the noisy or discretized data available to your program.

WORKED EXAMPLE

For x(t)=t*t, velocity is 2*t. At t=3, velocity is 6. The forward difference using h=0.1 is ((3.1)^2-3^2)/0.1=6.1, an approximation rather than the exact derivative.

PRACTICE + PASS CRITERION

Compare analytic and finite-difference derivatives of three functions over several step sizes. Pass: include units, an error table and an explanation of why the smallest tested h need not be best.

ASSIGNED STUDY

R08: differentiation, product rule and chain rule.

CHECK YOUR UNDERSTANDING

1. Differentiate x(t)=3*t*t.

2. What units does acceleration have when position is in meters?

3. Why can excessively small finite-difference steps be inaccurate?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 08.2 • MODULE 08

Integrals, accumulation and continuous probability

Learning goal Interpret integration as accumulated contribution.

An integral accumulates contributions over a domain. Integrating velocity over time gives displacement; integrating acceleration changes velocity. The fundamental theorem of calculus connects derivatives and integrals. A definite integral is a signed accumulation, so negative contributions can cancel positive ones.

Numerical quadrature approximates an integral with samples and weights. Rectangle, trapezoid and higher-order rules have different accuracy assumptions. More samples do not repair a wrong model or omitted discontinuity. In graphics, integration will later accumulate incoming light over directions; in probability, a density integrates to a probability over an interval.

WORKED EXAMPLE

With constant velocity 4 meters/second over 3 seconds, displacement is 12 meters. For v(t)=2*t from 0 to 3, integrating gives 9 meters, not the final velocity multiplied by the whole duration.

PRACTICE + PASS CRITERION

Implement rectangle and trapezoid integration for v(t)=2*t and a curved function. Pass: compare to an analytic integral, halve the step and report how error changes.

ASSIGNED STUDY

R08: integration and the fundamental theorem; R23: Monte Carlo Integration introduction.

CHECK YOUR UNDERSTANDING

1. Integrate constant velocity 5 over two seconds.

2. What is the integral of 2*t from 0 to 3?

3. Why can an integral be zero despite nonzero motion?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 08.3 • MODULE 08

Newtonian mechanics and differential equations

Learning goal Model motion from forces with clear simplifying assumptions.

Newton's second law relates net force to mass times acceleration. Position and velocity form the state of a basic particle model. Gravity, springs and damping provide forces with different dependencies. A differential equation specifies how state changes; initial conditions select a particular trajectory.

For a simple spring, force is proportional to displacement from rest and opposite its sign. Damping depends on velocity and dissipates energy in the model. Distinguish mass from weight and force from impulse. Games often choose simplified models for control and performance; simplification is legitimate when its behavior and limitations are explicit.

WORKED EXAMPLE

A 2 kg body under a constant 6 N net force has acceleration 3 m/s^2. Over 0.5 seconds, its velocity changes by 1.5 m/s. This says nothing about drag unless drag is included in the net force.

PRACTICE + PASS CRITERION

Implement a particle under constant acceleration and a damped spring. Pass: compare the constant-acceleration trajectory to its analytic solution and state what collision, rotation and air-resistance effects are omitted.

ASSIGNED STUDY

R28: simulation, bodies, forces and impulses; R08: differential-equation applications.

CHECK YOUR UNDERSTANDING

1. What acceleration results from 10 N on a 2 kg mass?

2. How does impulse differ from force?

3. Why must initial conditions accompany a differential equation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 08.4 • MODULE 08

Time integration, stability and error

Learning goal Choose and test a numerical update rather than assuming smaller code is correct.

Explicit Euler advances state using derivatives at the old state. Semi-implicit Euler updates velocity first and then uses the new velocity for position. Other methods trade more work for different accuracy and stability properties. A method can have small local error yet accumulate unacceptable global drift.

Timestep choice depends on the system, method and acceptable error. Stiff springs can require much smaller explicit steps than slow motion. Fixed timesteps aid repeatability but do not guarantee stability or identical results across machines. Compare against analytic cases, inspect energy where meaningful and repeat with a refined step before trusting a visually plausible animation.

WORKED EXAMPLE

For explicit Euler: x_next=x+v*dt; v_next=v+a*dt. For semi-implicit Euler, compute v_next first and use x_next=x+v_next*dt. The methods differ even with identical inputs.

PRACTICE + PASS CRITERION

Compare explicit and semi-implicit Euler on an undamped spring at three timesteps. Pass: plot or tabulate position and energy error, identify an unstable case, and avoid calling visual smoothness proof of physical accuracy.

ASSIGNED STUDY

R23: physically based animation topics via R19; R28: simulation stepping guidance.

CHECK YOUR UNDERSTANDING

1. Which velocity does semi-implicit Euler use for the position update?

2. Does a fixed timestep guarantee a stable simulation?

3. What is a useful check beyond watching the animation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 09

Probability, statistics and evidence

Prerequisites Modules 06 and 08.2.

Four lessons

09.1 Probability and conditional reasoning

Model uncertainty without confusing independent and conditional events.

09.2 Random variables, distributions and sampling

Distinguish an expected value from a guaranteed outcome.

09.3 Inference, uncertainty and benchmark design

Compare implementations without hiding variance or bias.

09.4 Regression, correlation and ethical telemetry

Fit a model while recognizing confounding and data limits.

Module project

Create a reproducible benchmark experiment and a loot-distribution simulation. Report assumptions, uncertainty, representative workloads and one result that did not support the original hypothesis.

Core references

R09 MIT OpenCourseWare — 18.05 Introduction to Probability and Statistics

R06 MIT OpenCourseWare — 6.042J Mathematics for Computer Science

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 09.1 • MODULE 09

Probability and conditional reasoning

Learning goal Model uncertainty without confusing independent and conditional events.

Probability assigns weights to events under a model. Conditional probability restricts attention to cases where another event occurred. Independence means learning one event does not change the probability of the other; it is an assumption to justify, not a default for every repeated observation.

Bayes' rule reverses conditioning using prior probabilities and evidence likelihoods. Rare-event base rates matter when interpreting an apparently accurate detector. In games, these ideas apply to randomized systems, telemetry and testing. A measured frequency approaches a model probability only under suitable sampling assumptions; a small sample can be very noisy.

WORKED EXAMPLE

For independent fair coin flips, the probability of two heads is 1/4. If the second outcome is deliberately copied from the first, the probability is 1/2, demonstrating why dependence changes the calculation.

PRACTICE + PASS CRITERION

Simulate independent and dependent binary events. Build a small contingency table for a hypothetical bug detector. Pass: calculate both P(alert|bug) and P(bug|alert) and explain why they differ.

ASSIGNED STUDY

R09: probability, conditional probability and Bayes' theorem.

CHECK YOUR UNDERSTANDING

1. What is the probability of two independent events with probabilities 0.5 and 0.2 both occurring?
2. Is P(A|B) generally equal to P(B|A)?
3. Why do rare-event base rates matter for an alert?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 09.2 • MODULE 09

Random variables, distributions and sampling

Learning goal Distinguish an expected value from a guaranteed outcome.

A random variable maps outcomes to numbers. Its expectation is a probability-weighted average; variance measures spread around that average. A distribution describes the possible values and their probabilities. Bernoulli, binomial and normal models have different assumptions and should not be chosen only because their names are familiar.

Pseudorandom generators are deterministic algorithms with internal state. A seed supports repeatable experiments, but a generator suitable for gameplay is not automatically suitable for security. Sampling can be biased by how data is collected. Use separate random streams when independent subsystems should not change each other's results merely by making extra draws.

WORKED EXAMPLE

A reward of 10 with probability 0.2 and 0 otherwise has expected value 2. A player still receives either 10 or 0 on one trial, not necessarily 2.

PRACTICE + PASS CRITERION

Implement a weighted reward sampler and compare observed frequencies to the intended distribution over several sample sizes. Pass: record seed, generator, counts and expected values; explain finite-sample variation.

ASSIGNED STUDY

R09: random variables, expectation and variance; R04: random reference.

CHECK YOUR UNDERSTANDING

1. What is the expectation of 20 with probability 0.25 and 0 otherwise?

2. Does an expectation of 5 guarantee a reward of 5 on a trial?

3. Why separate random streams for unrelated systems?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 09.3 • MODULE 09

Inference, uncertainty and benchmark design

Learning goal Compare implementations without hiding variance or bias.

A sample statistic estimates a property of a larger process. Confidence intervals express uncertainty under a sampling model; they are not a probability that a fixed frequentist parameter lies in a specific realized interval. Hypothesis tests require defined assumptions and do not measure practical importance by themselves.

For performance work, control workload, build mode, warmup and environmental changes. Use repeated independent runs where possible; adjacent frames are often correlated. Report distributions and tail latency rather than only an average. A statistically detectable difference may be too small to matter, while a rare long pause can be important despite little change in the mean.

WORKED EXAMPLE

Implementation A might average 5 ms but occasionally take 100 ms; B might average 6 ms with no large pauses in the tested sample. The preferred design depends on throughput and latency requirements, not the mean alone.

PRACTICE + PASS CRITERION

Compare two implementations using the same inputs, randomized test order and repeated runs. Pass: report median, a tail percentile, sample count and limitations; distinguish observations from causal claims.

ASSIGNED STUDY

R09: confidence intervals, hypothesis testing and experimental reasoning.

CHECK YOUR UNDERSTANDING

1. Why can adjacent frame times violate an independence assumption?

2. Does a small p-value establish a large useful improvement?

3. Why report a tail percentile alongside a mean?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 09.4 • MODULE 09

Regression, correlation and ethical telemetry

Learning goal Fit a model while recognizing confounding and data limits.

Regression relates an outcome to predictors under a chosen model. Correlation is association, not proof of causation. A confounder can influence both the apparent cause and the outcome. Models should be evaluated on held-out observations or a suitable validation scheme, not only on the data used to fit them.

Telemetry collection should have a defined purpose and minimize unnecessary personal data. For course projects, use synthetic or consented data rather than harvesting real players. Do not optimize only the easiest-to-measure metric. Session length, for example, is not equivalent to enjoyment, accessibility or wellbeing.

WORKED EXAMPLE

Players with better hardware may both buy more games and achieve higher frame rates. Observing a purchase/frame-rate correlation does not establish that purchases caused faster rendering.

PRACTICE + PASS CRITERION

Fit a simple performance model using entity count and measured update time. Hold out some workloads. Pass: report prediction error and one confounder; write a short data-minimization plan for the experiment.

ASSIGNED STUDY

R09: regression and inference; R01: ethics and data-management breadth.

CHECK YOUR UNDERSTANDING

1. Why is training-set fit alone insufficient?

2. What is a confounder?

3. Why should telemetry collect only what the question requires?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 10

Signals, optimization and numerical foundations

Prerequisites Modules 07-09.

Four lessons

10.1 Multivariable calculus and gradients

Use partial derivatives to reason about spatial fields and optimization.

10.2 Sampling, aliasing and Fourier ideas

Explain why discrete samples can misrepresent a continuous signal.

10.3 Convolution, filters and systems

Predict a filter's effect on data and latency.

10.4 Optimization, conditioning and approximation budgets

Choose an objective and distinguish convergence from correctness.

Module project

Complete Gate B: a mathematical engineering notebook plus tested transforms, numerical integration, signal filtering and an uncertainty-aware experiment.

Core references

R07 MIT OpenCourseWare — 18.06 Linear Algebra

R10 MIT OpenCourseWare — 6.003 Signals and Systems

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 10.1 • MODULE 10

Multivariable calculus and gradients

Learning goal Use partial derivatives to reason about spatial fields and optimization.

A multivariable function depends on several inputs. A partial derivative varies one input while holding the others fixed. The gradient collects partial derivatives and points toward steepest local increase under the usual Euclidean metric. The Jacobian describes the local linear behavior of a vector-valued function.

Gradients support terrain normals, inverse kinematics and parameter fitting. A gradient is local information, not a guarantee of the global best direction over a large step. Chain-rule reasoning must follow every transformation. Numerical derivatives are useful checks for analytic code, but choose perturbations that account for scale and rounding.

WORKED EXAMPLE

For f(x,y)=x*x+y*y, the gradient is (2*x,2*y). At (3,4), it is (6,8). Taking a small step in the negative gradient direction reduces f locally.

PRACTICE + PASS CRITERION

Derive gradients for three functions and check them with centered differences. Pass: test nonzero and stationary points, vary perturbation size and report errors rather than claiming exact equality.

ASSIGNED STUDY

R07: linearization foundations; R23: geometric differential quantities; use the program's multivariable extension reading.

CHECK YOUR UNDERSTANDING

1. What is the gradient of x*x + 3*y*y?

2. What does a Jacobian describe?

3. Does a zero gradient always identify a minimum?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 10.2 • MODULE 10

Sampling, aliasing and Fourier ideas

Learning goal Explain why discrete samples can misrepresent a continuous signal.

Sampling records a signal at discrete positions or times. Different high-frequency signals can produce the same samples, creating aliasing. Reconstruction needs assumptions about bandwidth and a suitable filter. For a strictly band-limited signal, the familiar sampling criterion requires a rate greater than twice its highest frequency; practical systems need transition bands and margin.

The Fourier viewpoint decomposes signals into frequency components. It connects audio, textures, animation and display artifacts. Increasing resolution helps only relative to the signal content. Filtering before downsampling removes frequencies that the lower rate cannot represent; filtering afterward cannot generally recover information already aliased.

WORKED EXAMPLE

A 900 Hz tone sampled at 1000 samples/second can appear as a 100 Hz tone. The samples do not uniquely preserve the original high-frequency oscillation.

PRACTICE + PASS CRITERION

Generate low- and high-frequency sine samples, then downsample a checker pattern with and without prefiltering. Pass: explain the artifacts using frequency content rather than calling them random noise.

ASSIGNED STUDY

R10: sampling and Fourier representation; R23: Sampling and Reconstruction.

CHECK YOUR UNDERSTANDING

1. What is the Nyquist frequency at 48,000 samples/second?

2. Why filter before downsampling?

3. Can post-filtering always undo aliasing?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 10.3 • MODULE 10

Convolution, filters and systems

Learning goal Predict a filter's effect on data and latency.

Convolution combines a signal with a kernel by weighted summation of shifted values. A moving average smooths high-frequency variation, but it also changes edges and can introduce delay depending on implementation. A linear time-invariant system is characterized by its impulse response under the model assumptions.

Boundary handling is part of a filter definition. Zero padding, wrapping and edge replication produce different results. Filters also have numeric and performance costs. In audio, a stable and bounded processing path is essential; in images, color-space assumptions matter when averaging values. Test an impulse and a constant signal before trusting a complex input.

WORKED EXAMPLE

A three-sample average of [3,6,9] is 6. Sliding this kernel across a sequence produces a smoothed sequence; values near either end require an explicit boundary policy.

PRACTICE + PASS CRITERION

Implement a one-dimensional convolution with documented boundary handling. Test an impulse, a constant and a step. Pass: predict the output before running and identify the filter's delay convention.

ASSIGNED STUDY

R10: convolution, impulse response and linear time-invariant systems.

CHECK YOUR UNDERSTANDING

1. What does convolving with an impulse reveal?

2. What is the average of 2, 5 and 8?

3. Why is boundary handling not an implementation footnote?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 10.4 • MODULE 10

Optimization, conditioning and approximation budgets

Learning goal Choose an objective and distinguish convergence from correctness.

Optimization minimizes or maximizes a specified objective under constraints. Gradient descent takes steps opposite the gradient; its behavior depends on step size, scaling and the shape of the objective. A local optimum need not be global. Constraints can be as important as the objective, such as keeping error below a visible threshold while reducing work.

Numerical engineering combines error sources: model approximations, finite precision, sampling and iterative termination. Define an error budget and a reference case. More iterations can reduce one error while leaving others unchanged. Stop according to a justified criterion and report failure to converge rather than silently returning a plausible number.

WORKED EXAMPLE

For f(x)=(x-3)^2, gradient descent uses x_next=x-alpha*2*(x-3). An excessively large alpha can oscillate or diverge even though the function has a simple minimum.

PRACTICE + PASS CRITERION

Implement gradient descent on a quadratic and a two-variable problem. Pass: compare step sizes, record residuals and state a stopping rule; complete Gate B with no unsupported claim of global optimality.

ASSIGNED STUDY

R07: least squares and conditioning; R23: numerical utilities; graduate optimization reading in the handbook.

CHECK YOUR UNDERSTANDING

1. Where is the minimum of (x-3)^2?

2. Why can reducing step size help an unstable iteration?

3. What should happen when an iterative method reaches its iteration limit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.